

Theorem 27.5

RANDOMIZED-MARKING has an expected competitive ratio of $O(\lg k)$ for the online caching problem with n requests and a cache of size k , against an oblivious adversary.

Before proving Theorem 27.5, we prove a basic probabilistic fact.

Lemma 27.6

Suppose that a bag contains $x + y$ balls: $x - 1$ blue balls, y white balls, and 1 red ball. You repeatedly choose a ball at random and remove it from the bag until you have chosen a total of m balls that are either blue or red, where $m \leq x$. You set aside each white ball you choose. Then, one of the balls chosen is the red ball with probability m/x .

Proof Choosing a white ball does not affect how many blue or red balls are chosen in any way. Therefore, we can continue the analysis as if there were no white balls and the bag contains just $x - 1$ blue balls and 1 red ball.

Let A be the event that the red ball is not chosen, and let A_i be the event that the i th draw does not choose the red ball. By equation (C.22) on page 1190, we have

$$\begin{aligned} \Pr\{A\} &= \Pr\{A_1 \cap A_2 \cap \cdots \cap A_m\} \\ &= \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \\ &\quad \Pr\{A_m \mid A_1 \cap A_2 \cap \cdots \cap A_{m-1}\}. \end{aligned} \quad (27.13)$$

The probability $\Pr\{A_1\}$ that the first ball is blue equals $(x - 1)/x$, since initially there are $x - 1$ blue balls and 1 red ball. More generally, we have

$$\Pr\{A_i \mid A_1 \cap \cdots \cap A_{i-1}\} = \frac{x - i}{x - i + 1}, \quad (27.14)$$

since the i th draw is from $x - i$ blue balls and 1 red ball. Equations (27.13) and (27.14) give

$$\Pr\{A\} = \left(\frac{x-1}{x}\right) \left(\frac{x-2}{x-1}\right) \left(\frac{x-3}{x-2}\right) \cdots \left(\frac{x-m+1}{x-m+2}\right) \left(\frac{x-m}{x-m+1}\right). \quad (27.15)$$

The right-hand side of equation (27.15) is a telescoping product, similar to the telescoping series in equation (A.12) on page 1143. The numerator of one term equals the denominator of the next, so that everything except the first denominator and last numerator cancel, and we obtain $\Pr\{A\} = (x - m)/x$. Since we actually want to compute $\Pr\{\bar{A}\} = 1 - \Pr\{A\}$, that is, the probability that the red ball is chosen, we get $\Pr\{\bar{A}\} = 1 - (x - m)/x = m/x$. ■

Now we can prove Theorem 27.5.

Proof We'll analyze RANDOMIZED-MARKING one epoch at a time. Within epoch i , any request for a block b that is not the first request for block b in epoch i must result in a cache hit, since after the first request in epoch i , block b resides in the cache and is marked, so that it cannot be evicted during the epoch. Therefore, since we are counting cache misses, we'll consider only the first request for each block within each epoch, disregarding all other requests.

We can classify the requests in an epoch as either old or new. If block b resides in the cache at the start of epoch i , each request for block b during epoch i is an *old request*. Old requests in epoch i are for blocks requested in epoch $i - 1$. If a request in epoch i is not old, it is a *new request*, and it is for a block not requested in epoch $i - 1$. All requests in epoch 1 are new. For example, let's look again at the request sequence in example (27.11):

1,2,1,5 4, 4, 1, 2, 4, 2 3,4,5 2, 2, 1, 2, 2 .

Since we can disregard all requests for a block within an epoch other than the first request, to analyze the cache behavior, we can view this request sequence as just

1,2,5 4,1,2 3,4,5 2, 1 .

All three requests in epoch 1 are new. In epoch 2, the requests for blocks 1 and 2 are old, but the request for block 4 is new. In epoch 3, the request for block 4 is old, and the requests for blocks 3 and 5 are new. Both requests in epoch 4 are new.

Within an epoch, each new request must cause a cache miss since, by definition, the block is not already in the cache. An old request, on the other hand, may or may not cause a cache miss. The old block is in the cache at the beginning of the epoch, but other requests might cause it to be evicted. Returning to our example, in epoch 2, the request for block 4 must cause a cache miss, as this request is new. The request for block 1, which is old, may or may not cause a cache miss. If block 1 was evicted when block 4 was requested, then a cache miss occurs and block 1 must be brought back into the cache. If instead block 1 was not evicted when block 4 was requested, then the request for block 1 results in a cache hit. The request for block 2 could incur a cache miss under two scenarios. One is if block 2 was evicted when block 4 was requested. The other is if block 1 was evicted when block 4 was requested, and then block 2 was evicted when block 1 was requested. We see that, within an epoch, each ensuing old request has an increasing chance of causing a cache miss.

Because we consider only the first request for each block within an epoch, we assume that each epoch contains exactly k requests, and each request within an epoch is for a unique block. (The last epoch might contain fewer than k requests. If it does, just add dummy requests to fill it out to k requests.) In epoch i , denote the number of new requests by $r_i \geq 1$ (an epoch must contain at least one new

request), so that the number of old requests is $k - r_i$. As mentioned above, a new request always incurs a cache miss.

Let us now focus on an arbitrary epoch i to obtain a bound on the expected number of cache misses within that epoch. In particular, let's think about the j th old request within the epoch, where $1 \leq j < k$. Denote by b_{ij} the block requested in the j th old request of epoch i , and denote by n_{ij} and o_{ij} the number of new and old requests, respectively, that occur within epoch i but before the j th old request. Because $j - 1$ old requests occur before the j th old request, we have $o_{ij} = j - 1$. We will show that the probability of a cache miss upon the j th old request is $n_{ij}/(k - o_{ij})$, or $n_{ij}/(k - j + 1)$.

Start by considering the first old request, for block $b_{i,1}$. What is the probability that this request causes a cache miss? It causes a cache miss precisely when one of the $n_{i,1}$ previous requests resulted in $b_{i,1}$ being evicted. We can determine the probability that $b_{i,1}$ was chosen for eviction by using Lemma 27.6: consider the k blocks in the cache to be k balls, with block $b_{i,1}$ as the red ball, the other $k - 1$ blocks as the $k - 1$ blue balls, and no white balls. Each of the $n_{i,1}$ requests chooses a block to evict with equal probability, corresponding to drawing balls $n_{i,1}$ times. Thus, we can apply Lemma 27.6 with $x = k$, $y = 0$, and $m = n_{i,1}$, deriving the probability of a cache miss upon the first old request as $n_{i,1}/k$, which equals $n_{ij}/(k - j + 1)$ since $j = 1$.

In order to determine the probability of a cache miss for subsequent old requests, we'll need an additional observation. Let's consider the second old request, which is for block $b_{i,2}$. This request causes a cache miss precisely when one of the previous requests evicts $b_{i,2}$. Let's consider two cases, based on the request for $b_{i,1}$. In the first case, suppose that the request for $b_{i,1}$ did not cause an eviction, because $b_{i,1}$ was already in the cache. Then, the only way that $b_{i,2}$ could have been evicted is by one of the $n_{i,2}$ new requests that precedes it. What is the probability that this eviction happens? There are $n_{i,2}$ chances for $b_{i,2}$ to be evicted, but we also know that there is one block in the cache, namely $b_{i,1}$, that is not evicted. Thus, we can again apply Lemma 27.6, but with $b_{i,1}$ as the white ball, $b_{i,2}$ as the red ball, the remaining blocks as the blue balls, and drawing balls $n_{i,2}$ times. Applying Lemma 27.6, with $x = k - 1$, $y = 1$, and $m = n_{i,2}$, we find that the probability of a cache miss is $n_{i,2}/(k - 1)$. In the second case, the request for $b_{i,1}$ does cause an eviction, which can happen only if one of the new requests preceding the request for $b_{i,1}$ evicts $b_{i,1}$. Then, the request for $b_{i,1}$ brings $b_{i,1}$ back into the cache and evicts some other block. In this case, we know that of the new requests, one of them did not result in $b_{i,2}$ being evicted, since $b_{i,1}$ was evicted. Therefore, $n_{i,2} - 1$ new requests could evict $b_{i,2}$, as could the request for $b_{i,1}$, so that the number of requests that could evict $b_{i,2}$ is $n_{i,2}$. Each such request evicts a block chosen from among $k - 1$ blocks, since the request that resulted in evicting $b_{i,1}$ did not also cause $b_{i,2}$ to be evicted. Therefore, we can apply Lemma 27.6, with $x = k - 1$,

$y = 1$, and $m = n_{i,2}$, and get that the probability of a miss is $n_{i,2}/(k-1)$. In both cases the probability is the same, and it equals $n_{ij}/(k-j+1)$ since $j = 2$.

More generally, o_{ij} old requests occur before the j th old request. Each of these prior old requests either caused an eviction or did not. For those that caused an eviction, it is because they were evicted by a previous request, and for those that did not cause an eviction, it is because they were not evicted by any previous request. In either case, we can decrease the number of blocks that the random process is choosing from by 1 for each old request, and thus o_{ij} requests cannot cause b_{ij} to be evicted. Therefore, we can use Lemma 27.6 to determine the probability that b_{ij} was evicted by a previous request, with $x = k - o_{ij}$, $y = o_{ij}$ and $m = n_{ij}$. Thus, we have proven our claim that the probability of a cache miss on the j th request for an old block is $n_{ij}/(k - o_{ij})$, or $n_{ij}/(k - j + 1)$. Since $n_{ij} \leq r_i$ (recall that r_i is the number of new requests during epoch i), we have an upper bound of $r_i/(k - j + 1)$ on the probability that the j th old request incurs a cache miss.

We can now compute the expected number of misses during epoch i using indicator random variables, as introduced in Section 5.2. We define indicator random variables

$$Y_{ij} = \mathbf{I}\{\text{the } j\text{th old request in epoch } i \text{ incurs a cache miss}\} ,$$

$$Z_{ij} = \mathbf{I}\{\text{the } j\text{th new request in epoch } i \text{ incurs a cache miss}\} .$$

We have $Z_{ij} = 1$ for $j = 1, 2, \dots, r_i$, since every new request results in a cache miss. Let X_i be the random variable denoting the number of cache misses during epoch i , so that

$$X_i = \sum_{j=1}^{k-r_i} Y_{ij} + \sum_{j=1}^{r_i} Z_{ij} ,$$

and so

$$\begin{aligned} \mathbf{E}[X_i] &= \mathbf{E}\left[\sum_{j=1}^{k-r_i} Y_{ij} + \sum_{j=1}^{r_i} Z_{ij}\right] \\ &= \sum_{j=1}^{k-r_i} \mathbf{E}[Y_{ij}] + \sum_{j=1}^{r_i} \mathbf{E}[Z_{ij}] \quad (\text{by linearity of expectation}) \\ &\leq \sum_{j=1}^{k-r_i} \frac{r_i}{k-j+1} + \sum_{j=1}^{r_i} 1 \quad (\text{by Lemma 5.1 on page 130}) \\ &= r_i \left(\sum_{j=1}^{k-r_i} \frac{1}{k-j+1} + 1 \right) \end{aligned}$$

$$\begin{aligned}
&\leq r_i \left(\sum_{j=1}^{k-1} \frac{1}{k-j+1} + 1 \right) \\
&= r_i H_k \quad \text{(by equation (A.8) on page 1142) ,} \quad (27.16)
\end{aligned}$$

where H_k is the k th harmonic number.

To compute the expected total number of cache misses, we sum over all epochs. Let p denote the number of epochs and X be the random variable denoting the number of cache misses. Then, we have $X = \sum_{i=1}^p X_i$, so that

$$\begin{aligned}
\mathbb{E}[X] &= \mathbb{E} \left[\sum_{i=1}^p X_i \right] \\
&= \sum_{i=1}^p \mathbb{E}[X_i] \quad \text{(by linearity of expectation)} \\
&\leq \sum_{i=1}^p r_i H_k \quad \text{(by inequality (27.16))} \\
&= H_k \sum_{i=1}^p r_i . \quad (27.17)
\end{aligned}$$

To complete the analysis, we need to understand the behavior of the optimal offline algorithm. It could make a completely different set of decisions from those made by RANDOMIZED-MARKING, and at any point its cache may look nothing like the cache of the randomized algorithm. Yet, we want to relate the number of cache misses of the optimal offline algorithm to the value in inequality (27.17), in order to have a competitive ratio that does not depend on $\sum_{i=1}^p r_i$. Focusing on individual epochs won't suffice. At the beginning of any epoch, the offline algorithm might have loaded the cache with exactly the blocks that will be requested in that epoch. Therefore, we cannot take any one epoch in isolation and claim that an offline algorithm must suffer any cache misses during that epoch.

If we consider two consecutive epochs, however, we can better analyze the optimal offline algorithm. Consider two consecutive epochs, $i-1$ and i . Each contains k requests for k different blocks. (Recall our assumption that all requests are first requests in an epoch.) Epoch i contains r_i requests for new blocks, that is, blocks that were not requested during epoch $i-1$. Therefore, the number of distinct requests during epochs $i-1$ and i is exactly $k + r_i$. No matter what the cache contents were at the beginning of epoch $i-1$, after $k + r_i$ distinct requests, there must be at least r_i cache misses. There could be more, but there is no way to have fewer. Letting m_i denote the number of cache misses of the offline algorithm during epoch i , we have just argued that

$$m_{i-1} + m_i \geq r_i . \quad (27.18)$$

The total number of cache misses of the offline algorithm is

$$\begin{aligned}
 \sum_{i=1}^p m_i &= \frac{1}{2} \sum_{i=1}^p 2m_i \\
 &= \frac{1}{2} \left(m_1 + \sum_{i=2}^p (m_{i-1} + m_i) + m_p \right) \\
 &\geq \frac{1}{2} \left(m_1 + \sum_{i=2}^p (m_{i-1} + m_i) \right) \\
 &\geq \frac{1}{2} \left(m_1 + \sum_{i=2}^p r_i \right) \quad (\text{by inequality (27.18)}) \\
 &= \frac{1}{2} \sum_{i=1}^p r_i \quad (\text{because } m_1 = r_1) .
 \end{aligned}$$

The justification $m_1 = r_1$ for the last equality follows because, by our assumptions, the cache starts out empty and every request incurs a cache miss in the first epoch, even for the optimal offline adversary.

To conclude the analysis, because we have an upper bound of $H_k \sum_{i=1}^p r_i$ on the expected number of cache misses for RANDOMIZED-MARKING and a lower bound of $\frac{1}{2} \sum_{i=1}^p r_i$ on the number of cache misses for the optimal offline algorithm, the expected competitive ratio is at most

$$\begin{aligned}
 \frac{H_k \sum_{i=1}^p r_i}{\frac{1}{2} \sum_{i=1}^p r_i} &= 2H_k \\
 &= 2 \ln k + O(1) \quad (\text{by equation (A.9) on page 1142}) \\
 &= O(\lg k) .
 \end{aligned}$$

■

Exercises

27.3-1

For the cache sequence (27.10), show the contents of the cache after each request and count the number of cache misses. How many misses does each epoch incur?

27.3-2

Show that LFU has a competitive ratio of $\Theta(n/k)$ for the online caching problem with n requests and a cache of size k .

27.3-3

Show that FIFO has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k .

27.3-4

Show that the deterministic MARKING algorithm has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k .

27.3-5

Theorem 27.4 shows that any deterministic online algorithm for caching has a competitive ratio of $\Omega(k)$, where k is the cache size. One way in which an algorithm might be able to perform better is to have some ability to know what the next few requests will be. We say that an algorithm is *l -lookahead* if it has the ability to look ahead at the next l requests. Prove that for every constant $l \geq 0$ and every cache size $k \geq 1$, every deterministic l -lookahead algorithm has competitive ratio $\Omega(k)$.

Problems
27-1 Cow-path problem

The Appalachian Trail (AT) is a marked hiking trail in the eastern United States extending between Springer Mountain in Georgia and Mount Katahdin in Maine. The trail is about 2,190 miles long. You decide that you are going to hike the AT from Georgia to Maine and back. You plan to learn more about algorithms while on the trail, and so you bring along your copy of *Introduction to Algorithms* in your backpack.² You have already read through this chapter before starting out. Because the beauty of the trail distracts you, you forget about reading this book until you have reached Maine and hiked halfway back to Georgia. At that point, you decide that you have already seen the trail and want to continue reading the rest of the book, starting with Chapter 28. Unfortunately, you find that the book is no longer in your pack. You must have left it somewhere along the trail, but you don't know where. It could be anywhere between Georgia and Maine. You want to find the book, but now that you have learned something about online algorithms, you want your algorithm for finding it to have a good competitive ratio. That is, no matter where the book is, if its distance from you is x miles away, you would like to be sure that you do not walk more than cx miles to find it, for some constant c . You do not know x , though you may assume that $x \geq 1$.³

² This book is heavy. We do not recommend that you carry it on a long hike.

³ In case you're wondering what this problem has to do with cows, some papers about it frame the problem as a cow looking for a field in which to graze.

What algorithm should you use, and what constant c can you prove bounds the total distance cx that you would have to walk? Your algorithm should work for a trail of any length, not just the 2,190-mile-long AT.

27-2 Online scheduling to minimize average completion time

Problem 15-2 discusses scheduling to minimize average completion time on one machine, without release times and preemption and with release times and preemption. Now you will develop an online algorithm for nonpreemptively scheduling a set of tasks with release times. Suppose you are given a set $S = \{a_1, a_2, \dots, a_n\}$ of tasks, where task a_i has **release time** r_i , before which it cannot start, and requires p_i units of processing time to complete once it has started. You have one computer on which to run the tasks. Tasks cannot be **preempted**, which is to say that once started, a task must run to completion without interruption. (See Problem 15-2 on page 446 for a more detailed description of this problem.) Given a schedule, let C_i be the **completion time** of task a_i , that is, the time at which task a_i completes processing. Your goal is to find a schedule that minimizes the average completion time, that is, to minimize $(1/n) \sum_{i=1}^n C_i$.

In the online version of this problem, you learn about task i only when it arrives at its release time r_i , and at that point, you know its processing time p_i . The offline version of this problem is NP-hard (see Chapter 34), but you will develop a 2-competitive online algorithm.

- a.** Show that, if there are release times, scheduling by shortest processing time (when the machine becomes idle, start the already released task with the smallest processing time that has not yet run) is not d -competitive for any constant d .

In order to develop an online algorithm, consider the preemptive version of this problem, which is discussed in Problem 15-2(b). One way to schedule is to run the tasks according to the shortest remaining processing time (SRPT) order. That is, at any point, the machine is running the available task with the smallest amount of remaining processing time.

- b.** Explain how to run SRPT as an online algorithm.
- c.** Suppose that you run SRPT and obtain completion times $C_1^P, \dots, C_n^P$. Show that

$$\sum_{i=1}^n C_i^P \leq \sum_{i=1}^n C_i^*,$$

where the C_i^* are the completion times in an optimal nonpreemptive schedule.

Consider the (offline) algorithm COMPLETION-TIME-SCHEDULE.

COMPLETION-TIME-SCHEDULE(S)

- 1 compute an optimal schedule for the preemptive version of the problem
- 2 renumber the tasks so that the completion times in the optimal preemptive schedule are ordered by their completion times
 $C_1^P < C_2^P < \dots < C_n^P$ in SRPT order
- 3 greedily schedule the tasks nonpreemptively in the renumbered order $a_1, \dots, a_n$
- 4 let $C_1, \dots, C_n$ be the completion times of renumbered tasks $a_1, \dots, a_n$ in this nonpreemptive schedule
- 5 **return** $C_1, \dots, C_n$

- d.* Prove that $C_i^P \geq \max \left\{ \sum_{j=1}^i p_j, \max \{r_j : j \leq i\} \right\}$ for $i = 1, \dots, n$.
- e.* Prove that $C_i \leq \max \{r_j : j \leq i\} + \sum_{j=1}^i p_j$ for $i = 1, \dots, n$.
- f.* Algorithm COMPLETION-TIME-SCHEDULE is an offline algorithm. Explain how to modify it to produce an online algorithm.
- g.* Combine parts (c)–(f) to show that the online version of COMPLETION-TIME-SCHEDULE is 2-competitive.

Chapter notes

Online algorithms are widely used in many domains. Some good overviews include the textbook by Borodin and El-Yaniv [68], the collection of surveys edited by Fiat and Woeginger [142], and the survey by Albers [14].

The move-to-front heuristic from Section 27.2 was analyzed by Sleator and Tarjan [416, 417] as part of their early work on amortized analysis. This rule works quite well in practice.

Competitive analysis of online caching also originated with Sleator and Tarjan [417]. The randomized marking algorithm was proposed and analyzed by Fiat et al. [141]. Young [464] surveys online caching and paging algorithms, and Buchbinder and Naor [76] survey primal-dual online algorithms.

Specific types of online algorithms are described using other names. *Dynamic graph algorithms* are online algorithms on graphs, where at each step a vertex or edge undergoes modification. Typically a vertex or edge is either inserted or

deleted, or some associated property, such as edge weight, changes. Some graph problems need to be solved again after each change to the graph, and a good dynamic graph algorithm will not need to solve from scratch. For example, edges are inserted and deleted, and after each change to the graph, the minimum spanning tree is recomputed. Exercise 21.2-8 asks such a question. Similar questions can be asked for other graph algorithms, such as shortest paths, connectivity, or matching. The first paper in this field is credited to Even and Shiloach [138], who study how to maintain a shortest-path tree as edges are being deleted from a graph. Since then hundreds of papers have been published. Demetrescu et al. [110] survey early developments in dynamic graph algorithms.

For massive data sets, the input data might be too large to store. *Streaming algorithms* model this situation by requiring the memory used by an algorithm to be significantly smaller than the input size. For example, you may have a graph with n vertices and m edges with $m \gg n$, but the memory allowed may be only $O(n)$. Or you may have n numbers, but the memory allowed may only be $O(\lg n)$ or $O(\sqrt{n})$. A streaming algorithm is measured by the number of passes made over the data in addition to the running time of the algorithm. McGregor [322] surveys streaming algorithms for graphs and Muthukrishnan [341] surveys general streaming algorithms.